

ore details can be found here.

Consequences

Not providing a way for users to build and run containers inside a workspace would hamper the value proposition of workspaces which aims to replace local development environments.

Leveraging Sysbox means we are dependent third-party tools for functionality. This is acceptable for reasons mentioned in 016: Allow users to run sudo commands inside a workspace.

Alternatives

Since we are already using Sysbox to provide secure way to run commands with sudo permissions inside a workspace, we decided to leverage it rather than find more alternatives to it.

SECTION: engineering/architecture/design-
documents/workspaces/decisions/018_remove_gitlab_workspaces_setup.md

Context

In 004: Authentication of user traffic to access a workspace it was decided to use GitLab Workspaces Proxy, a centralized proxy in the user's Kubernetes cluster, to authenticate and authorize user traffic to access a workspace.

Setting up GitLab Workspaces Proxy involves quite a lot of steps - buying/managing a domain, provisioning TLS certificates and setting them up for automatic renewal, creating an OAuth application, etc. Each of these step can go wrong in multiple ways thus complicating the setup.

We need a way to simplify the setup without burdening the user with buying/mangaing a domain, managing TLS certificates and other unrelated tasks to unlock the value of Workspaces for themselves. We want to reduce the barrier to entry for setting up Workspaces.

Decision

We will eliminate the need for GitLab Workspaces Proxy in the user's Kubernetes cluster by moving the logic of proxying traffic into the appropriate workspace to a server hosted and managed by GitLab as part of GitLab Agent Server(KAS). GitLab Agent for Workspaces(agentw) would be injected inside each workspace which will initiate a bi-directional gRPC connection with the above server. When user traffic arrives at the server, it will be authenticated, authorized

and proxied into the appropriate workspace using the bi-directional gRPC connection established earlier.

More details can be found [here](#).

Consequences

Since the need for GitLab Workspaces Proxy in the user's Kubernetes cluster is eliminated, users need not install any Ingress Controller.

This further reduces expectations from users and simplifies the overall architecture.

All traffic for the workspace will be through reverse gRPC tunnels established from within the workspace.

All workspaces will now be available on a GitLab managed domain.

Each workspace will continue to have a unique host.

Alternatives

We considered routing the user traffic for a workspace through Rails

to keep all the domain logic in the monolith but decided against it because Puma servers are blocked on each request.

Workspaces traffic would be a long persistent connection and thus it would become a scaling problem.

We considered routing the user traffic for a workspace through a new service - GitLab Workspaces Server

to keep all the domain logic in a service owned by the team but decided against it because we did not see any value over

keeping the logic in GitLab Agent Server/GitLab Agent especially because that project is a modularized monolith for such architectures outside of our Rails monolith.

SECTION: engineering/architecture/design-documents/work_items/_index.md

{{< engineering/design-document-header >}}

This document is a work-in-progress. Some aspects are not documented, though we expect to add them in the future.

Summary

Work Items is a new architecture created to support the various types of built and planned entities throughout the product, such as issues, requirements, and incidents. It will make these types easy to extend and customize while sharing the same core functionality.

Terminology

We use the following terms to describe components and properties of the Work items architecture.

Work Item

Base type for issue, requirement, test case, incident and task (this list is planned to extend in the future). Different work items have the same set of base properties but their widgets list is different.

Work Item types

A set of predefined types for different categories of work items. These are defined within the `workitemtypes` table in the database.

Work item types in the GitLab product

While the default work item types from above exist in the production database, they may not be used in the UI of the GitLab product since we are in the process of migrating each type from using legacy APIs to work items APIs. In cases where legacy items exist, we also need to migrate data to the issues table, where all work items are stored and build new widgets to encompass all functionality.

Work Item Type		Status in UI	
Availability in UI		Data migration to the issues table needed?	
Documentation			
-----	-----	-----	-----
-----	-----	-----	-----
-----	-----	-----	-----
Task	Implemented	Fully available	No
Task			
Objective	Implemented	Fully available behind a feature flag	No
Objective			
Key Result	Implemented	Fully available behind a feature flag	No
Key Result			
Incident	Planned	-	No
Incident			
Test case	Planned	-	No
Test case			
Requirement	Planned	-	No
Requirement			
Issue	In Development	-	No
Issue			
Epic	In Development	-	Yes, in progress
Epic			
Ticket	Planned	-	No
Ticket			

Work Item properties

Every Work Item type has the following common properties:

NOTE:

You can also refer to fields of Work Item to learn more.

- id - a unique Work Item global identifier;
- iid - internal ID of the Work Item, relative to the parent workspace (currently workspace can only be a project)
- Work Item type;
- properties related to Work Item modification time: createdAt, updatedAt, closedAt;
- title string;
- Work Item confidentiality state;
- Work Item state (can be open or closed);
- lock version, incremented each time the work item is updated;
- permissions for the current user on the resource
- a list of Work Item widgets

Work Item widgets

All Work Item types share the same pool of predefined widgets and are customized by which widgets are active on a specific type. The list of widgets for any certain Work Item type is currently predefined and is not customizable. However, in the future we plan to allow users to create new Work Item types and define a set of widgets for them.

Widget types (updating)

Widget	Description	Feature flag	Write permission	GraphQL Subscription Support
WorkItemWidgetAssignees	List of work item assignees	For epics work item type workitemsbeta, otherwise no	FF	Guest Yes
WorkItemWidgetAwardEmoji	Emoji reactions added to work item, including support for upvote/downvote counts	Anyone who can view	No	
WorkItemWidgetColor	Set color of a work item. Note: Color is available only for epics.	Reporter	No	
WorkItemWidgetCurrentUserTodos	User todo state of work item	Anyone who can view	No	
WorkItemWidgetDescription	Description of work item, including support for edited state, timestamp, and author	Reporter	No	
WorkItemWidgetDesigns	Design attachments for work items	Reporter	No	
WorkItemWidgetDevelopment	Show related branches and merge requests for work items	Reporter	No	
WorkItemWidgetHealthStatus	Health status assignment support for work item	Reporter	No	
WorkItemWidgetHierarchy	Hierarchy of work items, including support for boolean representing presence of children.	Guest	No	
WorkItemWidgetIteration	Iteration assignment support for work item	Reporter	No	
WorkItemWidgetLabels	List of labels added to work items, including support for checking whether scoped labels are supported	Reporter	Yes	
WorkItemWidgetLinkedItems	List of work items added as related to a given work item, with possible relationship types being relatesto, blocks, and blockedby. Includes support for individual counts of blocked status, blocked by, blocking, and related to.	Guest	No	
WorkItemWidgetMilestone	Milestone assignment support for work item	Reporter	No	
WorkItemWidgetNotes	List of discussions within a work item	Guest	Yes	
WorkItemWidgetNotifications	Notifications subscription status of a work item for current user	Anyone who can view	No	
WorkItemWidgetParticipants	Participants of a work item	Anyone who can view	No	

| WorkItemWidgetProgress | Progress value of a work item. Note: Progress is currently available only for OKRs. | okrmvc |Reporter|No|

| WorkItemWidgetRequirementLegacy | Legacy requirements | | |No|

| WorkItemWidgetRolledupDates | Set the start date and due date for epic work items, and roll up the start date and due date from child work items | |Reporter|No|

| WorkItemWidgetStartAndDueDate | Set start and due dates for a work item | |Reporter|No|

| WorkItemWidgetStatus | Status of a work item when type is Requirement, with possible status types being unverified, satisfied, or failed | | |No|

| WorkItemWidgetTestReports | Test reports associated with a work item | | | |

| WorkItemWidgetTimeTracking | Track total time spent on a work item | |Reporter|No|

| WorkItemWidgetWeight | Set weight of a work item | |Reporter|No|

| WorkItemWidgetLock | Lock/Unlock a work item | |Reporter|No|

Widget availability (updating)

| Widget | Epic | Issue | Task | Objective | Key Result |

|---|---|---|---|---|---|

| WorkItemWidgetAssignees | . | . | . | . | . | . |

| WorkItemWidgetAwardEmoji | . | . | . | . | . | . |

| WorkItemWidgetColor | . | . | . | . | . | . |

| WorkItemWidgetCurrentUserTodos | . | . | . | . | . | . |

| WorkItemWidgetDescription | . | . | . | . | . | . |

| WorkItemWidgetDesigns | . | . | . | . | . | . |

| WorkItemWidgetDevelopment | . | . | . | . | . | . |

| WorkItemWidgetHealthStatus | . | . | . | . | . | . |

| WorkItemWidgetHierarchy | . | . | . | . | . | . |

| WorkItemWidgetIteration | . | . | . | . | . | . |

| WorkItemWidgetLabels | . | . | . | . | . | . |

| WorkItemWidgetLinkedItems | . | . | . | . | . | . |

| WorkItemWidgetMilestone | . | . | . | . | . | . |

| WorkItemWidgetNotes | . | . | . | . | . | . |

| WorkItemWidgetNotifications | . | . | . | . | . | . |

| WorkItemWidgetParticipants | . | . | . | . | . | . |

| WorkItemWidgetProgress | . | . | . | . | . | . |

| WorkItemWidgetRequirementLegacy | . | . | . | . | . | . |

| WorkItemWidgetRolledupDates | . | . | . | . | . | . |

| WorkItemWidgetStartAndDueDate | . | . | . | . | . | . |

| WorkItemWidgetStatus | . | . | . | . | . | . |

| WorkItemWidgetTestReports | . | . | . | . | . | . |

| WorkItemWidgetTimeTracking | . | . | . | . | . | . |

| WorkItemWidgetWeight | . | . | . | . | . | . |

Legend

- . - Widget available
- . . - Widget planned to be available
- . - - Widget not available
- . - - Widget pending for consideration
- . - - Alternative widget planned

Work item relationships

Work items can be related to other work items in a number of different ways:

- Parent: A direct ancestor to the current work item, whose completion relies on completing this work item.
- Child: A direct descendant of the current work item, which contributes to this work item's completion.
- Blocked by: A work item preventing the completion of the current work item.
- Blocks: A work item whose completion is blocked by the current work item.
- Related: A work item that is relevant to the subject of the current work item, but does not directly contribute to or block the completion of this work item.

Hierarchy

Parent-child relationships form the basis of hierarchy in work items. Each work item type has a defined set of types that can be parents or children of that type.

As types expand, and parent items have their own parent items, the hierarchy capability can grow exponentially.

Currently, following are the allowed Parent-child relationships:

Type	Can be parent of	Can be child of	
-----	-----	-----	
Epic	Epic	Epic	
Issue	Task	Epic	
Task	None	Issue	
Objective	Objective	Objective	
Key result	None	Objective	

Work Item view

The new frontend view that renders Work Items of any type using global Work Item id as an identifier.

Task

Task is a special Work Item type. Tasks can be added to issues as child items and can be displayed in the modal on the issue view.

Feature flags

Since this is a large project with numerous moving parts, feature flags are being used to track promotions of available widgets. The table below shows the different feature flags that are being used, and the audience that they are available to.

feature flag name	audience	
--- ---		
workitems	defaulted to on	

| workitemsbeta | gitlab-org, gitlab-com |
| workitemsalpha | gitlab-org/plan-stage |

Contextual view feature flags

Feature flag name	Control area	Status
workitemsalpha	Child items in contextual view	Enabled for gitlab-org/plan-stage
epicslistdrawer	Epics list, epics board	Enabled for gitlab-org/plan-stage
issueslistdrawer	Issues list, issues board	Disabled

Issue work item view feature flags

Feature flag name	Control area	Status
workitemviewforissues	Enable work item view for issues	defaulted to on
workitemsviewpreference	Show toggle in the header section to enable or disable issue work item view	defaulted to on

For epic work item specific feature flags, please see the Epic Work Item Migration Epic.

Motivation

Work Items main goal is to enhance the planning toolset to become the most popular collaboration tool for knowledge workers in any industry.

- Puts all like-items (issues, incidents, epics, test cases etc.) on a standard platform to simplify maintenance and increase consistency in experience
- Enables first-class support of common planning concepts to lower complexity and allow users to plan without learning GitLab-specific nuances.

Goals

Scalability

Currently, different entities like issues, epics, merge requests etc share many similar features but these features are implemented separately for every entity type. This makes implementing new features or refactoring existing ones problematic: for example, if we plan to add new feature to issues and incidents, we would need to implement it separately on issue and incident types. With work items, any new feature is implemented via widgets for all existing types which makes the architecture more scalable.

Flexibility

With existing implementation, we have a rigid structure for issuable, merge requests, epics etc. This structure is defined on both backend and frontend, so any change requires a coordinated effort. Also, it would be very hard to make this structure customizable for the user without introducing a set of flags to enable/disable any existing feature. Work Item architecture allows frontend to

display Work Item widgets in a flexible way: whatever is present in Work Item widgets, will be rendered on the page. This allows us to make changes fast and makes the structure way more flexible. For example, if we want to stop displaying labels on the Incident page, we remove labels widget from Incident Work Item type on the backend. Also, in the future this will allow users to define the set of widgets they want to see on custom Work Item types.

A consistent experience

As much as we try to have consistent behavior for similar features on different entities, we still have differences in the implementation. For example, updating labels on merge request via GraphQL API can be done with dedicated `setMergeRequestLabels` mutation, while for the issue we call more coarse-grained `updateIssue`. This provides inconsistent experience for both frontend and external API users. As a result, epics, issues, requirements, and others all have similar but just subtle enough differences in common interactions that the user needs to hold a complicated mental model of how they each behave.

Work Item architecture is designed with making all the features for all the types consistent, implemented as Work Item widgets.

High-level architecture problems to solve

- how can we bypass groups and projects consolidation to migrate epics to Work Item type;
- dealing with parent-child relationships for certain Work Item types: epic > issue > task, and to the same Work Item types: issue > issue.
- implementing custom Work Item types and custom widgets

Links

- Work items initiative epic
- Tasks roadmap
- Work Item "Vision" Prototype
- Work Item Discussions

SECTION: engineering/architecture/design-documents/work_items_custom_fields/_index.md

<!-- Design Documents often contain forward-looking statements -->

<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->

{{< engineering/design-document-header >}}

Summary

This document outlines our approach to implementing a flexible custom fields system for work

items in GitLab.

We're extending standard work item fields/widgets by introducing customizable fields that allow teams to capture specialized information unique to their workflows.

The solution introduces user created custom fields that can be configured at the root group level and applied to various work item types. Premium and Ultimate users can create, manage, and use these custom fields across their groups, subgroups, and projects.

This initiative enables users to standardize how they record and report information, creating consistency across projects and supporting more powerful filtering and reporting capabilities.

Timeline & status updates

1. Custom Fields MVC1 was the Plan Stage's highest priority in FY26Q1 and it was introduced in GitLab 17.11 with the feature flag `customfieldsfeature`, which was enabled on GitLab.com, GitLab Self-Managed, and GitLab Dedicated.
2. The feature MVC1 became generally available in GitLab 18.0 with the feature flag removed.
3. MVC2 and MVC3 are pending prioritization and status updates can be found on the custom fields epic.

Glossary

Existing concepts and terms

1. Work Item Type: A classification that determines a work item's available features and behaviors through its associated widgets.
2. Widget: A functional component that provides specific capabilities to a work item type (for example "assignees" and "labels").
3. Namespace: In GitLab, a namespace is a unique name for a user or group that hosts projects.

New concepts and terms

1. Custom Field: A user-defined field that can be added to work items to capture specialized information beyond standard fields.
2. Field Type: The data type for a custom field, which determines its behavior and constraints (single-select, multi-select, number, text).
3. Select Option: A predefined value that can be selected in single-select or multi-select fields.
4. Custom Fields Widget: The component that displays custom fields and allows users to modify the values of a work item on the sidebar of the work item detail view.
5. Field Value: The data stored in a custom field for a specific work item.

Motivation

GitLab's standard fields provide a solid foundation for work item management, but many teams require additional specialized fields to track information unique to their workflows. This has been a long-requested feature documented in epic 235 and issues like 8988 dating back several years. The ability to enhance work items with custom fields enables organizations to tailor GitLab to their specific planning needs and creates consistency across projects.

Before this implementation, users had to rely on less structured approaches like using labels or storing information in the description, which limited filtering, reporting, and standardization capabilities. Custom fields solve these issues by providing structured data storage with different field types that can be consistently applied across work items.

Goals

1. Enable teams to add customized metadata to work items beyond standard fields
2. Provide different field types to accommodate various data requirements (text, number, single/multi-select)
3. Allow for consistent data collection and reporting across groups, subgroups, and projects
4. Improve filtering capabilities based on custom field values
5. Support standardization of workflows across teams
6. Use the work item architecture to extend custom field functionality to all supported work item types
7. Allow field configuration at the group level with inheritance to subgroups and projects

Non-Goals

1. Complex validation rules or dependencies between fields
2. Custom fields for entities outside the work items framework

Proposal

We propose developing a comprehensive custom fields system for work items in GitLab, centered on these key elements:

1. Custom fields are available for Premium and Ultimate customers.
2. Four field types are supported initially: single-select, multi-select, number, and text.
3. Fields are configured at the top-level group and are inherited by all subgroups and projects.
4. Fields can be assigned to specific work item types (issues, epics, etc.).
5. Field values live in the issues table.
6. Fields can be archived rather than deleted to preserve historical data.
7. The system uses the work item framework and widget concepts with the GraphQL API.

Note: This is an initial proposal, we plan to provide more configuration options and field types in the future.

Design and implementation details

This section details the core concepts, implementation architecture, and release plan for the work item custom fields system in GitLab.

Core Concepts

Custom Fields

Custom fields are defined at the top-level group and can be linked to multiple work item types. Each field has a specific type that defines its behavior and constraints:

- Single-select: Users can select one option from a predefined list
- Multi-select: Users can select multiple options from a predefined list
- Number: Users can enter a numeric value
- Text: Users can enter free-form text (limited to 1024 characters)

Custom fields can be active or archived. Archived fields preserve historical data but are no longer available for new work items or edits.

There's a limit of 50 active custom fields per top-level group.

Select Options

For single-select and multi-select fields, select options define the available choices. Options have a name and position which determines their display order.

A single-select or multi-select field can have at most 50 select options.

Field Values

Field values store the actual data for each custom field on a work item. Different value types are used depending on the field type:

- TextFieldValue: Stores string values for text fields
- NumberFieldValue: Stores numeric values for number fields
- SelectFieldValue: Stores the selected option(s) for single-select and multi-select fields

Field Association with Work Item Types

Custom fields are associated with specific work item types through a many-to-many relationship. This allows for different work item types to have different sets of fields.

A work item type can have at most 10 custom fields assigned to it.

Implementation Architecture

Database Schema

For custom field configuration

```
mermaid
erDiagram
    namespaces ||--o{ customfields : ""
    customfields o|--o{ customfieldselectoptions : ""
    workitemtypes ||--o{ workitemtypecustomfields : ""
    customfields }|--|{ workitemtypecustomfields : ""
    customfields {
        int id
        int namespaceid
        string name
        enum fieldtype
```

```

    timestamp archivedat
}
customfieldselectoptions {
    int id
    int namespaceid
    int customfieldid
    string value
}
workitemtypecustomfields {
    int id
    int namespaceid
    int customfieldid
    int workitemtypeid
}

```

Notes:

- fieldtype can be select, multi-select, numeric, text, etc..
- customfieldselectoptions is used to store the options for select / multi-select types
- This is decoupled from work item widget definitions so that it is easier to add to the default types and so it can be reused for MRs and custom objects in the future.

For storing work item custom field values

```

mermaid
erDiagram
    workitems ||--o{ workitemselectfieldvalues : ""
    workitems ||--o{ workitemnumericfieldvalues : ""
    workitems ||--o{ workitemtextfieldvalues : ""
    workitemselectfieldvalues {
        int id
        int namespaceid
        int workitemid
        int customfieldid
        int customfieldoptionsid
    }
    workitemnumericfieldvalues {
        int id
        int namespaceid
        int workitemid
        int customfieldid
        numeric value
    }
    workitemtextfieldvalues {
        int id
        int namespaceid
        int workitemid
        int customfieldid
        text value
    }

```

```
}
```

Notes:

- Each custom field type uses its own table so that we can have foreign keys to maintain referential integrity.

Models and Associations

The custom fields system is implemented with the following key models:

- Issuables::CustomField: Stores the field definition, including name and field type
- Issuables::CustomFieldSelectOption: Stores options for single-select and multi-select fields
- WorkItems::TypeCustomField: Links custom fields to work item types
- WorkItems::TextFieldValue, WorkItems::NumberFieldValue, WorkItems::SelectFieldValue: Store field values for specific work items

Custom Fields Widget

We use the CUSTOMFIELDS widget to display and manage custom field values in work items. This widget is added to all work item types that support custom fields.

The widget renders different input components based on the field type:

- Text fields use a text input component
- Number fields use a numeric input component
- Single-select and multi-select fields use a dropdown component

API Design

The feature uses the existing work items GraphQL API by extending the work item widget framework, and returns custom fields data from the CUSTOMFIELDS widget.

To retrieve metadata about available fields, we can query the widget definitions endpoint, specifying both the work item type and namespace context.

Here's an example of the GraphQL query to fetch all the custom fields from a top-level group:

```
graphql
query groupCustomFields($fullPath: ID!, $active: Boolean!) {
  group(fullPath: $fullPath) {
    id
    customFields(active: $active) {
      count
      nodes {
        id
        name
        fieldType
        active
      }
    }
  }
}
```

```

    createdAt
    updatedAt
    selectOptions {
      id
      value
    }
    workItemTypes {
      id
      name
    }
  }
}
}
}

```

Variables example:

```

graphql
{
  "fullPath": "gitlab-org",
  "active": true
}

```

Here is an example of the GraphQL query to fetch custom fields for a work item:

```

graphql
query namespaceWorkItem($fullPath: ID!, $iid: String!) {
  workspace: namespace(fullPath: $fullPath) {
    id
    workItem(iid: $iid) {
      id
      widgets {
        type
        ... on WorkItemWidgetCustomFields {
          type
          customFieldValues {
            customField {
              id
              name
              fieldType
            }
          }
          ... on WorkItemNumberFieldValue {
            value
          }
          ... on WorkItemTextFieldValue {
            value
          }
          ... on WorkItemSelectFieldValue {

```

```
selectedOptions {
  id
  value
}
```

Variables example:

```
graphql
{
  "fullPath": "gitlab-org",
  "iid": "235"
}
```

Permissions

- To create, edit, archive and unarchive a custom field, you must have at least the Maintainer role for the group.
- To set custom field values for a work item, you must have at least the Planner role for the work item's project or group. If you have the Guest role, you can set custom fields only when creating a work item.

Managing Custom Fields

On the top-level group settings page, users can configure custom fields to make them available for work items in that group, its subgroups, and projects.

On that page, you can also define for which work item type you want that custom field to be related. Once a custom field is created and linked to a work item type, it should be visible on that work item type page.

To learn how to manage a top-level group's custom fields, check the guide.

Archiving Custom Fields

Rather than deleting custom fields, we support archiving them to preserve historical data. When a field is archived:

1. It is marked with an archivedat timestamp
2. It no longer appears in the field selection UI
3. It's no longer available for new work items
4. Existing field values are preserved for historical reference

Fields can be unarchived to make them available again. To learn how to archive or unarchive custom fields, check the guide.

Filtering by Custom Fields

Users can filter work items by custom field values on group and project list pages. This is implemented in some using the existing search and filter capabilities, with extensions for custom field types:

- Text fields: Search for work items with specific text content
- Number fields: Filter by numeric value
- Single-select fields: Filter by selected option
- Multi-select fields: Filter by one selected option (currently)

Filtering currently available on the following pages:

- Group/Issues list (Example)
- Group/Issues boards (Example)
- Project/Issues list (Example)
- Project/Issues boards (Example)

Feature Flags and Licensed Feature

We used the feature flag `customfieldsfeature` throughout the development of this feature. The feature flag was removed in GitLab 18.0 when the feature became generally available.

Since the feature is only available in Premium and Ultimate tier, we consider it a licensed feature. The feature name is `customfields`.

Implementation and release plan

We've identified these phases for this initiative:

MVC1: Core Functionality and basic fields (GitLab 17.11)

- Implement the database schema for custom fields
- Create the GraphQL API for managing custom fields
- Add work item type associations to control which work item types show which fields
- Build the UI for creating and managing custom fields at the group level
- Implement the custom fields widget for work item detail pages
- Implement the custom fields widget for work item create pages
- Support for all four field types (text, number, single-select, multi-select)
- Basic filtering capabilities for issues lists and boards
- Implement archiving and unarchiving of custom fields
- System notes for custom field changes
- Removal of the feature flag, making the feature generally available

Next iterations

- MVC1 enhancements

- MVC2 initial plan: Date custom fields, enable updating custom fields via quick action and other enhancements (Epic)
- MVC 3 initial plan: Smart fields based on existing GitLab objects as field values (Epic)

Alternative Solutions

Do nothing and continue to use labels

Pros:

- No development cost
- Labels are already familiar to users
- Labels can be color-coded

Cons:

- Labels are not structured data
- Labels are less powerful for filtering and reporting
- Labels don't support different data types (numbers, select lists, etc.)
- Labels don't allow for validation or formatting
- Labels are not organized by work item type

Decision Registry

1. Use EAV data model to store custom field values. The main advantage is referential integrity and type validation.
2. Custom fields should not depend on custom work item types. We should be able to create custom fields on the existing default types.
3. Limit configuration to root namespaces. Configuration applies to all subgroups and descendant projects. Configuration of custom fields for subgroups is planned for future iterations.
4. Initial custom field types: Select field, number and text input
5. Rather than deleting custom fields, we decided to support archiving them to preserve historical data.
6. Custom fields filtering was not added to the epics board due to prioritization. It should work by default once the epics board is updated to use work items.

Resources

1. Top level epic for this initiative (235)
2. Custom fields guide
3. GitLab 17.11 release announcement

Team

Please mention the current team in all MRs related to this document to keep everyone updated. We don't expect everyone to approve changes.

text

@gweaver @donaldcook @nickleonard @fernanda.toledo @psimyn @engwan @stefanosxan

SECTION: engineering/architecture/design-documents/work_items_custom_status/_index.md

<!-- Design Documents often contain forward-looking statements -->

<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->

{{< engineering/design-document-header >}}

Summary

This document outlines our approach to implementing a flexible status system for work items in GitLab.

We're evolving beyond the binary open/closed states and label-based status tracking by introducing proper status fields with customization capabilities.

The solution introduces system-defined statuses as the foundation for Premium and Ultimate users, with users able to add statuses, change status name or color, and alter the order of statuses. We'll initially target Tasks before expanding to Issues, Epics and other work item types, while Free users will continue using the binary open/closed state system.

This initiative lays the groundwork for users to more effectively manage work item lifecycles and addresses problems caused by requiring labels for status management.

Timeline & status updates

1. Custom Status is the Plan Stage's highest priority in FY26Q2, and we've planned to deliver the first two iterations to General Availability by the 18.2 milestone
1. Status updates can be derived from the custom statuses epic and the iteration child epics.

Glossary

Existing concepts and terms

1. Work Item Type: A classification that determines a work item's available features and behaviors through its associated widgets.
1. Widget: A functional component that provides specific capabilities to a work item type (for example "assignees" and "labels").
1. State: The fundamental binary classification of a work item as either 'open' or 'closed'.

New concepts and terms

1. Status: A specific step in a work item's workflow ("In progress", "Done", "Won't do")

that belongs to a category and maps to a binary state (open/closed).

1. Status category: A logical grouping for statuses (triage, todo, inprogress) that determines their effect on a work item's state (and icon).
1. System-defined status: A system-provided status that cannot be modified and that is available to get started with work item statuses.
1. Custom Status: A namespace-defined status that replaces system-defined statuses for all groups and projects within that namespace.
1. Lifecycle: A collection of statuses that can be applied to a work item type. It allows statuses to be grouped into meaningful workflows that can be reused consistently across types and namespaces.
1. Status Widget: The component that displays status and allows users to modify the status of a work item on list, board, and detail views.

Motivation

GitLab currently relies on a combination of labels and two formal states (open/closed) to represent work item status.

This approach has significant limitations for teams managing complex workflows.

The binary open/closed system doesn't provide enough context about where items stand in a process, and overusing

labels for status tracking creates inconsistency and management overhead.

Goals

1. Expand work item status beyond the binary open/closed system to represent more workflow stages
1. Improve context on the status of work items
1. Enable better reporting on the reasons for closing issues (completed vs. duplicated/moved/won't do)
1. Enhance lead/cycle time calculations by distinguishing active time from waiting time
1. Build on top of our work item system to leverage status for all work item types
1. Reduce reliance on labels for tracking status

Non-Goals

1. Make status usable beyond the work items framework (for example for other entities outside the framework).
1. Force every status-like functionality into work item status (for example incident paging status and requirements verification status)

Proposal

We propose implementing a flexible status system for work items in GitLab, built around these core concepts:

1. Status is available for Premium and Ultimate customers.
1. Free customers continue to use state (open/closed) and labels.

1. Customizable workflows through lifecycles that can be applied to work item types and that hold statuses.

1. System-defined statuses as a starting point.

1. Allow users to create and modify status and lifecycles as custom statuses and custom lifecycles.

1. Sync state and status along the workflow to ensure consistency and compatibility.

1. Use the work item framework and widget concepts and the GraphQL API.

1. Migration tools to assist users in transitioning from workflow labels to the new status system.

Design and implementation details

This section outlines the core concepts, implementation architecture, and rollout plan for the new work item status system in GitLab.

We'll explore the key components and detail how they interact to create a flexible, scalable status management solution.

Core Concepts

Lifecycles

Lifecycles function as a holder for statuses and can be applied to work item types. They define which statuses are available for items of a given work item type.

For example: In a customers root group, issues should use the "Engineering" lifecycle, tasks "Kanban", and epics, objectives, and key results the "Strategy" lifecycle.

All lifecycles reuse the "Done" and "Won't do" statuses, but overall the number of statuses and their position is different.

We start with providing one default lifecycle for work item types like tasks, issues, epics, objectives and key results.

Service Desk tickets may use a different lifecycle with different statuses like waiting for first response,

waiting for customer and waiting for next response. But that is to be defined.

See this discussion to learn more.

There's a limit of 50 lifecycles per namespace.

Statuses

A status can be used in multiple lifecycles and can be attached to a work item based on the available lifecycle of

the work item type and namespace. The default lifecycle contains these statuses:

- To do
- In progress
- Done
- Won't do
- Duplicate

There's a limit of max. 70 statuses per namespace. A max. of 30 statuses can be attached to a lifecycle.

Status Categories

Statuses are organized into categories that are not user-configurable that determine their behavior:

```
ruby
CATEGORIES = {
  triage: 1, Exists but without system-defined status
  todo: 2,
  inprogress: 3,
  done: 4,
  canceled: 5
}.freeze
```

The category is only visible in the management area of statuses and not in list, detail and board views.

The category defines the icon of the status.

Status-State Relationship and Transitions

Status builds on top of the existing state system (open and closed).

We need to transition work items automatically when either state or status change according to the following rules:

- Statuses in done and canceled categories automatically set work items to closed state
- All other categories maintain work items in open state
- The lifecycle defines default transition statuses (this is true for both system-defined and custom lifecycles):
 - Default Open Status: Applied when creating and reopening items
 - Default Closed Status: Applied when closing items
 - Default Duplicated Status: Applied when marking items as duplicates (or moved, promoted TBD)

Implementation Architecture

Distinction between system-defined and custom entities

We've decided against prepopulating statuses for each root namespace, as this approach would make modifying or extending statuses more challenging and significantly increase the database table size. Instead, we're introducing system-defined statuses and lifecycles as defaults, which can then be customized into custom statuses and lifecycles.

The Cells initiative aims to make our platform horizontally scalable and resilient. To achieve this, we need a sharding key in every database table.

Sharding keys cannot be null, and we're discouraged from using a custom sequence (such as using -1 for system-defined statuses or lifecycles, or reserving the first 1,000 IDs for system-defined entities).

Because system-defined data doesn't belong to a namespace and can be considered global, we need to handle it separately.

For all other tables, we'll use namespaceid as the sharding key.

We considered three options for managing system-defined entities:

1. Define system-defined entities in code (chosen approach)
2. Store system-defined data in a separate table
3. Use a single table and reserve first 1000 IDs with a custom CHECK constraint

We've chosen to implement the first option: defining system-defined entities in code. This approach offers the best balance of simplicity, scalability, and maintainability for our system.

Fixed items models and associations

Since system-defined data is static per definition we can hard-code it and benefit from not hitting the database.

The challenge is that we cannot make joins and use ActiveRecord methods.

We introduce the concept of fixed items models that include a module that adds ActiveRecord-like methods and defines the static data in an array of hashes called ITEMS.

- MR that introduces the abstraction modules
- MR that introduces the concrete models and associations

For example:

```
ruby
class StaticModel
  include ActiveModel::Model
  include ActiveModel::Attributes
  include ActiveRecord::FixedItemsModel::Model

  ITEMS = [
    {
      id: 1,
      name: 'To do'
    }
  ]

  attribute :id, :integer
  attribute :name, :string
end
```

ActiveRecord-like methods can be used like:

```
ruby
StaticModel.find(1)
StaticModel.where(name: 'To do')
StaticModel.findby(name: 'To do')
StaticModel.all
```

To make associations possible we introduce a custom association `belongstofixeditems` which resolves associations to fixed items models like this (id column needs to be present):

```
ruby
class MyModel < ApplicationRecord
  include ActiveRecord::FixedItemsModel::HasOne

  belongstofixeditems :staticmodel, fixeditemsclass: StaticModel
end
```

The association can then be used as:

```
ruby
m = MyModel.last
m.staticmodel Returns fixed items model instance
m.staticmodel = StaticModel.find(1)
m.staticmodelid = 1 still possible
m.staticmodel? Bool
m.save! Saves association
```

Overview of classes

```
plantuml
@startuml
class WorkItem
class WorkItems::Statuses::CurrentStatus {
  namespaceid: Sharding key
  workitemid
  systemdefinedstatusid
  customstatusid
}
class WorkItems::Statuses::Custom::Status {
  namespaceid: Sharding key
  name
  color
  category
  description
```

```

}
class WorkItems::Statuses::Custom::Lifecycle {
  namespaceid: Sharding key
  name
  defaultopenstatusid
  defaultclosedstatusid
  defaultduplicatestatusid
}
class WorkItems::Statuses::Custom::LifecycleStatus {
  namespaceid: Sharding key
  lifecycleid
  statusid
  position
}
class WorkItems::Statuses::SystemDefined::Status {
  name
  color
  category
  position
}
class WorkItems::Statuses::SystemDefined::Lifecycle {
  name
  workitembasetypes: Array
  statusids: Array
  defaultopenstatusid
  defaultclosedstatusid
  defaultduplicatestatusid
}
class WorkItems::Type
class WorkItems::TypeCustomLifecycle {
  namespaceid: Sharding key
  workitemtypeid
  lifecycleid
}

```

WorkItem -- WorkItems::Statuses::CurrentStatus

WorkItems::Statuses::CurrentStatus -- WorkItems::Statuses::Custom::Status

WorkItems::Statuses::CurrentStatus -- WorkItems::Statuses::SystemDefined::Status

WorkItems::Statuses::Custom::Status -- WorkItems::Statuses::Custom::LifecycleStatus

WorkItems::Statuses::Custom::LifecycleStatus -- WorkItems::Statuses::Custom::Lifecycle

WorkItems::Statuses::Custom::Lifecycle -- WorkItems::TypeCustomLifecycle

WorkItems::TypeCustomLifecycle -- WorkItems::Type

WorkItems::Statuses::SystemDefined::Status -- WorkItems::Statuses::SystemDefined::Lifecycle

WorkItems::Statuses::SystemDefined::Lifecycle -- WorkItems::Type

@enduml

To encapsulate everything status related we use the `WorkItems::Statuses` namespace.

Starting from the `WorkItem` we'll create a join model called `WorkItems::Statuses::CurrentStatus` that holds the status associations of a work item. We have separate columns `systemdefinedstatusid` and `customstatusid` which map to the different concepts of how we store system-defined and custom data. The model itself abstracts that away and we just set and get a status (using `status` and `status=`).

We can determine on the root namespace level whether all descendants of the root namespace use system-defined statuses or custom statuses.

For lists that collect work items from various root namespaces we won't check which status to use but instead check for the availability of data on the join model. If `customstatusid` is set, use the custom status. If not use the system-defined status. To efficiently fetch this data for work item lists, we use a bulk status resolver which only adds two additional queries. One to load the join model and another to load custom statuses.

We use the fields `defaultopenstatusid`, `defaultclosedstatusid`, and `defaultduplicatestatusid` to make automatic state/status transitions possible.

When the assigned status for any of these changes, we won't change status for existing work items.

Instead we'll use the newly assigned status for new transitions or new items only.

Namespace Configuration

Each root namespace exclusively uses either system-defined statuses or custom statuses. When editing statuses for the first time, the system creates copies of all system-defined statuses as custom statuses.

All work items of this namespace and its descendants will need to be migrated from the old to the new statuses (which will happen in the background). After that there's no way back to using system-defined statuses for this namespace.

Lifecycles and statuses can only be configured on the root namespace level in the first iterations.

It's to be defined how the mechanic to configure on a lower level will work and which restrictions will apply.

In dashboards and lists that render items from different root namespaces:

Statuses with the same name across different namespaces will be grouped in frontend filters (for example in dashboards).

The API call will include a filter with the grouped statuses in a OR condition.

For example: we filter for status Done which is available in two root namespaces (A and B). The frontend filter only shows Done, but the API call will filter for A::Done OR B::Done.

API design

We use the existing work items GraphQL API and build on top of the work items widgets concept.

The API returns status related data of a work item in the STATUS widget.
We can get a list of available statuses for a given work item type in a namespace by querying the widget definitions for the work item type.

We'll add concrete queries once the widget API is finalized.

Permissions

We've decided not to introduce new permissions for work item statuses. Instead, authorization is handled by existing work item permissions like readworkitem or updateworkitem.

This approach avoids redundant permission checks by leveraging GraphQL's higher-level query execution for authorization, improving query performance by reducing the number of Permission checks.

Additionally, work item status-specific resolvers like BulkStatusResolver and AllowedStatusesResolver ensure that the licensed feature is available and the feature flag is enabled before proceeding.

Status widget

We use the STATUS widget.

We already introduced the mock API using the widget name CUSTOMSTATUS because STATUS was already taken.

This legacy widget represents the verification status of requirements and should be renamed to VERIFICATIONSTATUS.

We marked both widgets and fields as experiment in 17.9, so we can rename them like this:

1. STATUS --> VERIFICATIONSTATUS (see MR)
1. CUSTOMSTATUS --> STATUS (see MR)

Backfill status data for existing work items

Backfill System Defined Status

Each work item of a work item type that supports status should have a status assigned. We'll backfill the systemdefinedstatusid on workitemcurrentstatuses table, before adding status support for a work item type.

To conserve database storage we'll only backfill status data for open work items.

Although custom status is a licensed feature, we will backfill status data for all work items of a given work item type regardless of license.

We will also perform automatic status transitions for all items, including those that are open, closed, or marked as duplicates.

For example a newly created work item will receive the default open status, and when closed, it will transition to the default closed status. Or when a closed work item without a status would be reopened it would transition to default open status.

See the discussion on this topic on this issue.

This approach ensures:

1. Status is immediately available when a namespace adds a license.
1. Work items maintain correct status assignments when a namespace changes tiers.

This significantly reduces complexity by eliminating the need for additional data migrations during namespace tier changes.

Backfill Custom Statuses (backup option)

Alternatively, as a backup, we've discussed the option of backfilling custom statuses records, with system defined values.

In this case we'd need to backfill not only `workitemcurrentstatuses`, but also for each root level Group we'd need to populate data in:

- `workitemcustomstatuses` - 5 records per root Group
- `workitemcustomlifecycles` - 1 record per root Group
- `workitemcustomlifecyclestatuses` - 5 records per each `workitemcustomlifecycles` record
- `workitemtypecustomlifecycles` - 2 records(Issue and Task) initially, per each `workitemcustomlifecycles` record.

This results in more database storage used from the start.

The benefit being that we would not require an on-demand status migration from system defined statuses to custom statuses, see Option 1 in Status migration and migration wizard section.

Status migration and migration wizard

We need to migrate statuses of work items in the following cases:

1. A namespace transitions from system-defined statuses to custom statuses
1. A user applies a different lifecycle to a work item type
1. A user creates statuses from labels or scoped labels
1. A status will be deleted and its assigned work items need to be migrated to a new status

We'll persist a mapping from one status to another in the database and run a job in the background

that updates the status association.

Optionally we'll also consider the mapping in list queries and include the old and new status during the migration.

We acknowledge that there might be a short time where status data is inconsistent in list views during migration.

This is especially true for namespaces with a large number of work items.

Details about the database structure and service architecture are to be defined.

Namespaces downgrade to free tier

We discussed implications of downtiering from using custom statuses to system-defined statuses.

The main takeaway is that it would introduce a decent amount of complexity to handle mapping existing custom statuses to previously existing system-defined statuses.

Because of that we decided that status will only be available in Premium and Ultimate tier.

When a customer transitions to the free tier, they won't see status anymore.

But we keep all status relevant data and associations and continue to do status/state transitions.

If they decide to uptier again, they'll see their custom statuses again and all work items will be in a correct status.

Other status-like functionality

In GitLab we have entities with functionality that is comparable to work item status but represent something different

or are highly custom integrations for a specific purpose. While we have ideas on how to include these into work item

statuses, we refrain from doing so in the foreseeable future. Specifically we've evaluated this for:

1. Incident paging status. We plan to relabel this in the UI from "Status" to something else and when we migrate

incidents to work items, we'll introduce this in a separate widget for now.

See this discussion for details

1. Requirements verification status. We're renaming the existing STATUS widget to VERIFICATIONSTATUS

and keep the functionality separate.

Challenges

1. Because the incidents detail view won't be migrated to the work item detail view short-term, status won't be available for this work item type.

It's to be defined whether we'll work on an intermittent solution or postpone adding status to incidents until the migration is completed.

See this discussion for details

Feature flags and licensed feature

We'll use the feature flag `workitemstatusfeatureflag` throughout the development of this feature.

The actor needs to be the root group.

For testing purposes, the feature flag is currently enabled in production for the Plan Stage testing group called `gl-demo-ultimate-plan-stage`.

We're using this feature flag rollout issue.

Since the feature will only be available in Premium and Ultimate tier, we consider it a licensed feature.

The feature name is `workitemstatus`.

The name differs from the feature flag because we cannot use the same name.

Implementation and release plan

We've identified these iterations for this initiative:

Iteration 1 (internal dogfooding)

- Iteration 1 epic
- Implement system-defined status and join model
- Make the status widget visible on Tasks only
- Implement status setting and viewing functionality
- Implement state/status transitions
- Add `/status` quick action

Iteration 2 (GA)

- Iteration 2 epic
- Implement custom statuses
- Expand support to Issues
- Board integration (Issues only)
- Filter by a single status on list views (if ready only work item list, else legacy list, no support for legacy epic list)
- Status management (create, update, reorder, delete)

Iteration 2 is the GA release. The `workitemstatusfeatureflag` feature flag will be used for internal testing and then dogfooding on `gitlab-org` and `gitlab-com`.

Nice to have

- Migration from labels to statuses

Iteration 3 (fast follow)

- Iteration 3 epic
- Expand support to epics: epic detail view, epic list view, legacy epic board view. If the new board experience becomes available, skip implementing the legacy epic board view and focus on the new experience instead.

Iteration 4

- Iteration 4 epic
- Customizable status for all work item types
- Lifecycle management (create, update, assign work item types, delete)
- Status available on dashboards (more complex because they can be fed by different root namespaces)
- Calculations for milestone and iteration burndown

Future iterations

- Future iterations epic
- Customize lifecycles / statuses at any hierarchy level (group, subgroup project)

Internationalization

System-defined statuses will only use english names.
Customers can customize statuses to match their preferred language.

Alternative Solutions

Do nothing and continue to use state and labels

Pros:

- No change to existing workflows
- No development cost

Cons:

- Doesn't address customer feedback about label overuse
- Doesn't provide a first-class status integration into the product
- Limits reporting capabilities

Decision registry

This section documents key architectural and implementation decisions made during the development of this feature.

1. Define system-defined entities in code rather than database tables.
1. Status will only be available in Premium and Ultimate tier.
1. No new permissions for work item statuses. Reuse readworkitem and updateworkitem.
1. Use the name STATUS widget for custom status. Rename existing STATUS widget to VERIFICATIONSTATUS and rename CUSTOMSTATUS widget to STATUS.

1. Statuses are unique across the namespace and are attached to a lifecycle.
We don't create new statuses for each lifecycle. So the status done may be attached to multiple lifecycles.
 1. System-defined statuses won't be internationalized.
We only use english names.
 1. We'll dogfood iteration 1 internally.
and use the release plan outlined in this document.
 1. We'll backfill only open work items
with a default open status.
 1. We'll always add status data regardless of license to eliminate the need for additional data migrations during tier changes.
 1. Once a namespace uses custom statuses, there's no way back to system-defined statuses.
 1. When the default open/closed/duplicate status of a lifecycle is changed, it only affects new transitions and new item creations.
 1. We decided on limits: max. 70 statuses and
50 lifecycles per namespace and 30 statuses per lifecycle.
 1. We'll set the statuses for all items of supported work item types only if the feature flag state and license are enabled.
 1. We'll show the default open status as a preselected value on the work item create form.
 1. We'll be implementing work item status badge and filters in legacy issues list.
 1. Expanding support to epics, including the epic detail view, epic list view,
and legacy epic board view will be included in Iteration 3 (Fast follow). If the new board experience is available by the time of implementation, we'll skip the legacy board view and focus on the new experience instead.
 1. Backfill Custom Statuses is added as a backup option if later on we determine that migration from system-defined statuses to custom statuses poses more challenges than initially foreseen.
 1. As part of Iteration 2, we'll only allow the deletion of custom statuses that are not in use.
- Statuses that have already been assigned to a work item, have an associated status mapping or are set as one of the default statuses (open, closed, duplicate) in a lifecycle can still be updated, but not deleted.

Resources

1. Top level epic for this initiative
1. Designs for list/detail/board
1. Designs for status management
1. Designs for status migration wizard
1. Initial spike work for first iteration
1. Proof of concept (POC) for system-defined and custom statuses

Team

Please mention the current team in all MRs related to this document to keep everyone updated.
We don't expect everyone to approve changes.

text

@gweaver @nickleonard @donaldcook @ntepluhina @msaleiko @aslota @deepika.guliani @stefanosxan
@psimyn @engwan

Feel free to mention the following people to spread the word:

text

@johnhope @amandarueda @caitlinsteele

SECTION: engineering/architecture/guidelines/_index.md

SECTION: engineering/architecture/guidelines/database.md

GitLab is a single application that relies on a single data store (terminology we use in the context of stringent durability requirements, e.g., PostgreSQL as opposed to Redis).

This directive drives GitLab's product development: while GitLab is SaaS first, it is not SaaS only. In terms of databases, GitLab supports self-managed configurations with a single database cluster, generally through the use of logical databases. Only at scale do we contemplate unfolding the single cluster into multiple ones, as GitLab.com does. This implies that the code base stays stable and can leverage flexible backend strategies to address scalability. Self-managed and GitLab.com will continue to operate as a single application, and self-managed in particular will continue to provide simplicity while offering the flexibility to scale.

There is no fundamental aversion to multiple data stores, but we want to avoid the pitfalls of having too many of them, especially when a variety of data engines are involved. Additional data stores introduce complexity in terms of management and development. Using new or adopting existing databases will involve tradeoffs, which have to be meticulously managed at scale. We have placed some gatekeepers before a separate data store can be deployed, and the following guidelines are intended to help you determine what the best course of action is and how to proceed.

These guidelines rely heavily on the Database Working Group glossary.

We need a new database

We need a new database is becoming a more common request, and while it seems simple, it is deceptively so, particularly for new services and new teams.

First and foremost, access to the database (any database) can only happen within the confines of the production environment and relevant access controls and procedures. Features and services requesting new resources must undergo a production readiness review.

Second, we need to fully understand what that statement actually means, particularly in terms of the type of data that has to be stored and queried, how tightly coupled it is to the main database, and how schemas and configurations are to be managed.

Third, we need to understand what it will mean in terms of resource utilization (space requirements, transaction rates, expected growth for both).

Finally, we need to understand if the data requires a new data store engine, since this will have significant implications once it is deployed in production in terms of the deployment itself, on-going maintenance (such as upgrades), observability integrations, and, more importantly, troubleshooting during incidents.

We use a relational database (PostgreSQL) as the primary metadata storage engine. Additionally, we also use a data structure store (Redis), which, while critical to the functioning of GitLab, stores more dynamic data.

Relational: PostgreSQL

PostgreSQL is our relational database engine of choice, and, until recently, all metadata for projects, issues, merge requests, users, and so on resided in a single cluster. Its schema is managed by the Rails application (db/structure.sql). We commonly refer to this data store as the main database. As scalability needs increased, we adopted some best practices to address them. Some items have been extracted from the database (diffs were the first ones), while others were deployed in separate logical databases or instances (at scale) because they were already separate services and had low data coupling requirements (Registry and Praefect). Still others have been decomposed (CI) because of their scale (and because we found ways to work around tight coupling).

Data Structure: Redis

Redis is used for specialized, non-relational needs, including queues (Sidekiq jobs marshal jobs into JSON payloads), persistent state (session data and exclusive leases), and cache (repository data such as branch and tag names, and view partials). Redis is a critical component of the application, but it is not used with the same durability constraints as PostgreSQL.

Considerations

Four basic questions have to be answered to determine the appropriate database:

1. What kind of code base comprises the application?
1. What kind of data does the application need to store?
1. What kinds of queries does the application need to run against the database?
1. What resource utilization is expected, both today and in the near future, in terms of space consumed, transaction rates (for read and writes), and latencies?

The answers to the first three questions are likely known in advance, since they are really baked into the application; the last one tends to be a little less clear, and while exact answers are not necessary, ballpark estimates (for both at deployment time and peering a bit into the future) are required. Specialized teams, such as Database, Scalability, and Memory can aid in uncovering some of these unknowns. In any event, the answers to these questions should avoid specific technologies (with some exceptions). As an additional input, the scaling model for GitLab.com based on business expectations and past growth can be consulted to understand future scaling requirements.

Default answer: the main Postgres database

Traditionally, the default answer has been to use our main database (PostgreSQL). However, we are probably at a point where this can no longer be the case, primarily for three reasons:

1. We have to carefully manage load on the main database for GitLab.com, particularly in terms of transaction rates, connections, and memory utilization, to ensure